Data-Driven Modelling Report Lab 2

Béguelin Laurent
Master in Electrical Engineering
Université Libre de Bruxelles (ULB)
Brussels, Belgium
Laurent.beguelin@ulb.be

Loic Carel Tchakounte Tchatat
Master in Electrical Engineering
Université Libre de Bruxelles (ULB)
Brussels, Belgium
loic.carel.tchakounte.tchatat@ulb.be

Diel Donald Chedjou Fotsing
Master in Electrical Engineering
Université Libre de Bruxelles (ULB)
Brussels, Belgium
diel.donald.chedjou.fotsing@ulb.be

I. Introduction

The objective of this laboratory is to design, implement, and evaluate a two-layer Artificial Neural Network (ANN). The network is designed to solve a multi-class classification problem: identifying three distinct species of Iris flowers (Iris setosa, Iris versicolor, and Iris virginica) based on four morphological measurements. The core focus centers on building a fundamental understanding of forward propagation, categorical cross-entropy loss computation, matrix-based backpropagation, and parameter optimization via batch gradient descent.

II. Dataset & Pre-processing

The Iris dataset comprises 150 observations (50 per species), each described by four continuous features: sepal length, sepal width, petal length, and petal width. Features are standardized to zero mean and unit variance. Class labels are converted to one-hot vectors, representing setosa, versicolor, and virginica as $[1, 0, 0]$, $[0, 1, 0]$, and $[0, 0, 1]$ respectively, to match the three-output softmax layer. The dataset is split into 80% training (120 samples) and 20% test (30 samples).

III. Network Architecture & Mathematical Formulation

The baseline network follows the lab-prescribed architecture, consisting of three layers as illustrated below:

$$4 \xrightarrow{\text{input}} 8 \xrightarrow{\sigma} 3 \xrightarrow{\text{softmax}} \hat{y}$$

Input Layer: Receives the feature matrix $X \in \mathbb{R}^{m\times 4}$, where m is the number of samples and the four columns correspond to sepal length, sepal width, petal length, and petal width.
Hidden Layer: Contains 8 neurons. It applies a learnable weight matrix $W^{(1)} \in \mathbb{R}^{4\times 8}$ and bias $b^{(1)} \in \mathbb{R}^{1\times 8}$ to the input, followed by a sigmoid activation function $\sigma(z) = \frac{1}{1+e^{-z}}$, which squashes each neuron output to the range $(0, 1)$. The number of hidden neurons (8) was specified by the lab and provides sufficient representational capacity for this low-dimensional problem.
Output Layer: Contains 3 neurons, one per class. It applies weights $W^{(2)} \in \mathbb{R}^{8\times 3}$ and bias $b^{(2)} \in \mathbb{R}^{1\times 3}$, followed by a softmax activation that normalizes the raw scores into a valid probability distribution, where each row sums to 1 and each entry represents the predicted probability of a given class.

A. Forward Propagation & Cost Function

The forward pass maps X through two affine transformations separated by a sigmoid activation, producing class probabilities via softmax. The categorical cross-entropy loss then quantifies the average prediction error, penalizing confident wrong predictions heavily:

$$Z^{(1)} = XW^{(1)} + b^{(1)}, \quad A^{(1)} = \sigma(Z^{(1)}) \tag{1}$$

$$Z^{(2)} = A^{(1)}W^{(2)} + b^{(2)}, \quad A^{(2)} = \frac{e^{Z^{(2)}_{i,k}}}{\sum_{j=1}^{3} e^{Z^{(2)}_{i,j}}} \tag{2}$$

$$\mathcal{L} = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{3} Y_{ik} \log A^{(2)}_{ik} \tag{3}$$

IV. Training & Backpropagation Implementation

The purpose of backpropagation is to evaluate the analytical gradients of the loss function with respect to every weight and bias parameter using the mathematical chain rule. Working backward from the output layer:

- Output Error: The function evaluates the difference between the predicted probabilities and the true one-hot targets ($A^{(2)} - Y$) to determine the final layer error.
- Output Parameter Gradients: This error is matrix-multiplied by the transposed hidden activations to calculate the gradients for the second-layer weights ($dW^{(2)}$) and summed across samples for the bias ($db^{(2)}$)
- Hidden Error: The output error is projected backward through the second weight matrix and multiplied by the first derivative of the Sigmoid function to find the hidden layer error ($dZ^{(1)}$).
- Input Parameter Gradients: Finally, the input features are matrix-multiplied by the hidden error to output the final gradients for the first-layer weights ($dW^{(1)}$) and biases ($db^{(1)}$).

The training loop runs for 2000 full epochs using a fixed learning rate of $\eta = 0.1$. Using batch gradient descent, the gradient descent function simultaneously updates all weights and biases by subtracting a small fraction of their computed gradients, controlled by the fixed learning rate ($\eta = 0.1$). Over 2000 epochs, this iterative reduction shifts the network's internal parameters toward the global loss minimum.

V. Results & Discussion

Plotting the stored loss arrays across 2000 epochs for three learning rates reveals clearly distinct convergence behaviours. The most conservative rate ($\eta = 0.01$) fails to reach a competitive loss within 2000 epochs, while the project-suggested baseline ($\eta = 0.1$) converges steadily, stabilizing near 0.10. The hypertuned model ($\eta = 0.5$) converges fastest and reaches the lowest final loss, without exhibiting the oscillations or divergence typically associated with large step sizes, likely due to the well-conditioned nature of the Iris dataset.

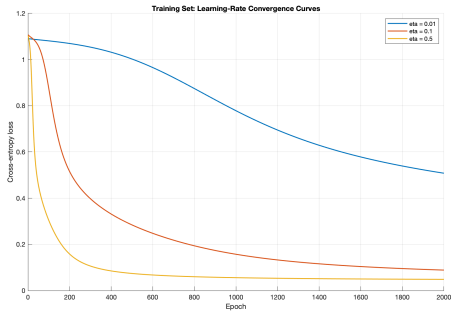

Fig. 1: Training loss curve

Evaluation Metrics

Four metrics are reported. **Accuracy** gives the overall fraction of correct predictions. **Precision** and **Recall** measure, per class, the proportion of predicted positives that are correct and the proportion of actual positives that are retrieved, respectively. The **F1-score**, their harmonic mean, provides a single balanced indicator robust to class imbalance.

Baseline Results

The trained network achieves strong performance on both sets. *Iris setosa* is perfectly classified (36/36) owing to its clear separation from the other two species, while misclassifications are strictly confined to the *versicolor*/*virginica* boundary. This is confirmed by the pairwise decision boundaries in Figure 2, where *setosa* forms a consistently isolated cluster, whereas *versicolor* and *virginica* overlap, particularly in sepal dimensions.

Hyperparameter Tuning

Three hyperparameters were varied independently. Replacing sigmoid with ReLU or tanh yielded no consistent improvement, as vanishing gradients are not a limiting factor in this shallow, low-dimensional network. Adding a second hidden layer similarly provided no benefit, as a single layer of 8 neurons already offers sufficient capacity for this problem. The learning rate was the only parameter with a measurable impact: $\eta = 0.01$ slowed convergence without accuracy gains, while $\eta = 0.5$ achieved the lowest final loss and fastest convergence, demonstrating that the lab-suggested $\eta = 0.1$ is slightly conservative for this dataset.

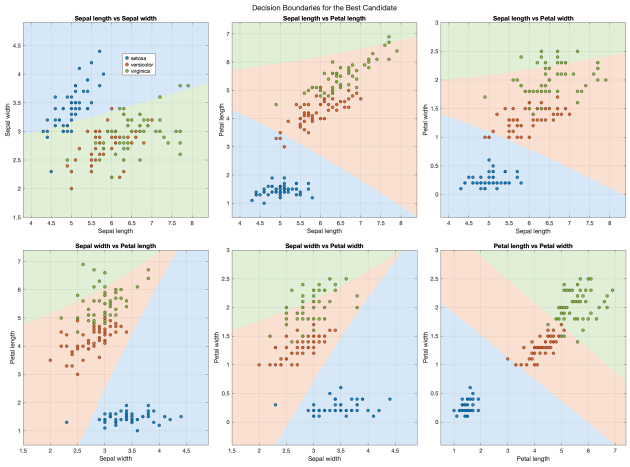

Fig. 2: Decision boundaries for all six feature pairs. *Iris setosa* is consistently well-separated; *Iris versicolor* and *Iris virginica* overlap in several subspaces.

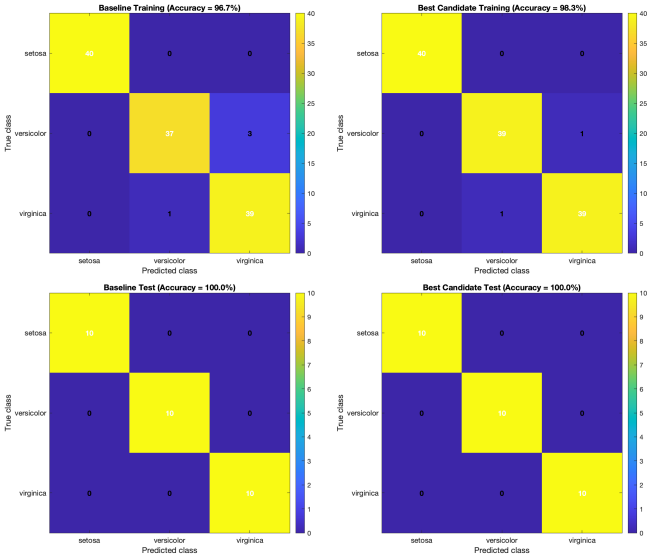

Fig. 3: Confusion matrices for the baseline and hypertuned models on training and test sets.

As shown in Figure 3, both the baseline and hypertuned models achieve perfect generalisation on the held-out test set, with macro accuracy, precision, recall, and F1-score all reaching 100%.

VI. Conclusion

The implemented two-layer ANN achieves perfect classification on the held-out test set under both configurations. While promising, this result should be interpreted with caution as the test set comprises only 30 samples, limiting the statistical significance of the evaluation. We thanks Ali Al-Zawqari for this valuable project.